Réaliser une application informatique complète

RAPPORT FINAL

Licence Informatique - 3ème Année

COMME Matthieu

FETTAH Sofiane

Année universitaire 2025-2026

1. Introduction 3
2. Analyse des écarts 4
2.1 Objectifs techniques 4
2.1.1 Jeu en réseau 4
2.1.2 Éléments cachés aux autres 4
2.1.3 Déconnexion et reconnexion 4
2.1.4 Tests fonctionnels 4
2.1.5 Architecture logicielle adaptée 5
2.1.6 Intelligence Artificielle et Bots 5
2.2 Objectifs organisationnels 5
2.3 Analyse des réussites et difficultés 5
2.3.1 Réussites 5
2.3.2 Difficultés 6
3. Retour d'expérience 6
3.1 Matthieu Comme - Backend 6
3.2 Sofiane Fettah - Frontend 7
4. Conclusion 7
Annexes 8
1. Déploiement en ligne 8
2. Dépôt du code source 8
3. Diagramme de classes backend 9
4. Diagramme de classes Frontend 10

1. Introduction

Ce document dresse le bilan de notre projet de jeu UNO multijoueur en réseau. Il ne s'attarde pas sur les règles du jeu, mais se concentre sur l'évaluation du travail accompli, les écarts avec le cahier des charges initial et notre retour d'expérience technique. Nous nous sommes réparti les tâches de la manière suivante :

Matthieu Comme: conception et développement du serveur backend (Spring Boot, logique métier, WebSockets).

Sofiane Fettah : conception et développement de l'interface frontend (React, interface utilisateur, animations).

2. Analyse des écarts

2.1 Objectifs techniques

2.1.1 Jeu en réseau

L'application implémente le multijoueur en temps réel via WebSockets et le protocole STOMP. Le serveur autorise la création de parties configurables via `maxPlayers`. Le routage des messages (`/topic/game/{gameId}/{playerId}`) permet de synchroniser l'état de la partie entre tous les clients connectés simultanément.

2.1.2 Éléments cachés aux autres

La séparation des informations privées et publiques est garantie par le serveur. L'objet `PlayerDTO` distribué à tous les adversaires ne contient que des données publiques, incluant le nombre de cartes mais pas leur valeur. La méthode `mapToGameStateDTO` filtre spécifiquement la main du joueur en fonction de l'identifiant du destinataire.

2.1.3 Déconnexion et reconnexion

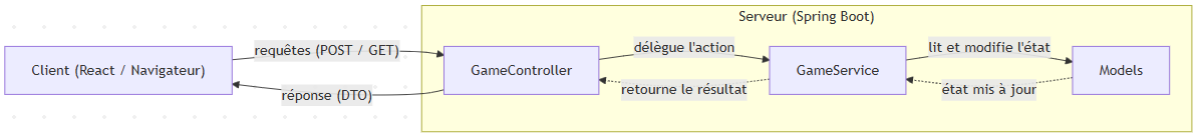
Un système de reconnexion gère les coupures réseau côté client. Lorsqu'un joueur est déconnecté, un bot le remplace. Si le joueur retourne sur l'application, l'affichage lui propose de reprendre la partie en cours.

2.1.4 Tests fonctionnels

L'architecture découplée favorise la testabilité. La logique métier principale étant isolée dans des classes indépendantes du réseau (`Game`, `Deck`, `Card`…), les tests unitaires peuvent valider le déroulement des tours sans nécessiter le lancement d'un serveur

2.1.5 Architecture logicielle adaptée

Le projet respecte une architecture en couches standard:



GameController : intercepte les requêtes REST et gère la diffusion STOMP.

GameService: centralise la logique et manipule les modèles.

Models: encapsule les règles strictes du jeu et l'état des objets

DTO (Data Transfer Object): allège et sécurise les données envoyées sur le réseau.

2.1.6 Intelligence Artificielle et Bots

Le système intègre des joueurs autonomes pour garantir qu'une partie puisse toujours démarrer et se terminer, quel que soit le nombre de joueurs humains. Le serveur ajoute des bots pour atteindre le nombre de joueurs requis dans le salon. Si un joueur humain se déconnecte, un bot prend le relai, évitant de bloquer la partie.

2.2 Objectifs organisationnels

L'ambition technique (découverte de Spring Boot, React et des WebSockets) a exigé un investissement important. L'objectif a été tenu, même si on a passé bien plus de temps que prévu sur le projet. La séparation stricte des dossiers frontend et backend a evité les conflits de versionnement sous Git et a permis un travail asynchrone efficace.

2.3 Analyse des réussites et difficultés

2.3.1 Réussites

L'encapsulation stricte côté back-end et le format d'échange JSON universel ont rendu la synchronisation front/back fiable. L'esthétique du client offre un rendu final compréhensible et agréable. Nous avons créé une API REST pour permettre au client et au serveur de communiquer.

Le jeu est disponible en ligne, le back est déployé sur Render, et le front sur Vercel.

2.3.2 Difficultés

La gestion de la déconnexion et de la reconnexion a été un beau défi. Nous l'avons résolu en stockant dans le localStorage du navigateur les identifiants de la partie et du joueur au début de la partie. S'il quitte et tente de se reconnecter, le serveur vérifie s'il existe dans cette partie un joueur avec cet identifiant qui est déconnecté. Si c'est le cas, il le réintègre en lui envoyant l'état de la partie.

3. Retour d'expérience

3.1 Matthieu Comme - Backend

Ce projet a consolidé mes compétences en architecture logicielle côté serveur.

J'ai appris à implémenter une communication via WebSockets. Cela m'a confronté à la gestion de la diffusion des messages : multicast pour l'état public, unicast pour les mains privées.

J'ai appliqué le concept de ne jamais faire confiance au client. Toute la validation des règles du UNO est centralisée dans le GameService. J'ai mis en pratique le design pattern DTO pour sérialiser en JSON seulement les informations nécessaires à l'affichage, rendant la triche par inspection réseau impossible.

J'ai dû gérer les cas limites des jeux en réseau, comme les déconnexions soudaines. J'ai implémenté des listeners pour repérer les pertes de connexion et mis en place un système remplaçant le joueur déconnecté par un bot pour ne pas bloquer la partie.

J'ai renforcé ma capacité à découpler les responsabilités. Séparer le conteneur de la logique interne des models a rendu le moteur de jeu testable unitairement et évolutif. L'usage d'énumérations a permis de sécuriser le typage de l'application.

3.2 Sofiane Fettah - Frontend

Ce projet a été ma première vraie expérience avec un framework JavaScript moderne, et honnêtement React m'a d'abord déstabilisé. Penser l'interface comme une fonction de l'état plutôt que de manipuler le DOM directement demandait un changement de logique complet. J'ai fini par comprendre cette mécanique en l'appliquant concrètement sur la partie de jeu : chaque mise à jour du serveur déclenche un nouveau rendu, et React s'occupe de synchroniser l'affichage automatiquement.

La partie la plus enrichissante a été la connexion WebSocket. J'ai dû gérer l'abonnement par joueur pour que chacun ne reçoit que ses propres cartes, implémenter une reconnexion automatique avec tentatives progressives, et distinguer les cas où il faut du polling HTTP de ceux où le WebSocket suffit. Cette logique m'a appris à raisonner sur le flux de données en temps réel plutôt que sur de simples requêtes classiques.

J'ai également pris conscience que la sécurité côté frontend ne se résume pas à cacher des éléments visuels. Ne jamais exposer les cartes adverses, filtrer les entrées utilisateur, poser des verrous sur les actions critiques pour éviter les doublons — autant de réflexes que je n'aurais pas eus naturellement avant ce projet. Couplé à la contrainte d'un hébergement limité qui m'a forcé à réduire les appels réseau au strict nécessaire, j'ai développé une sensibilité à l'optimisation que je n'avais pas en arrivant.

4. Conclusion

Le produit final est une application web complète, asynchrone et sécurisée. L’architecture découplée, associée à un hébergement en ligne, constitue une base de projet solide. Les futures évolutions pourraient inclure :

- la persistance des profils en base de données avec des statistiques avancées
- un système de classement avec elo
- d’autres modes de jeux avec des variantes de règles

Annexes

1. Déploiement en ligne

Jouer au jeu (Frontend hébergé via Vercel) : https://uno-upjv.vercel.app

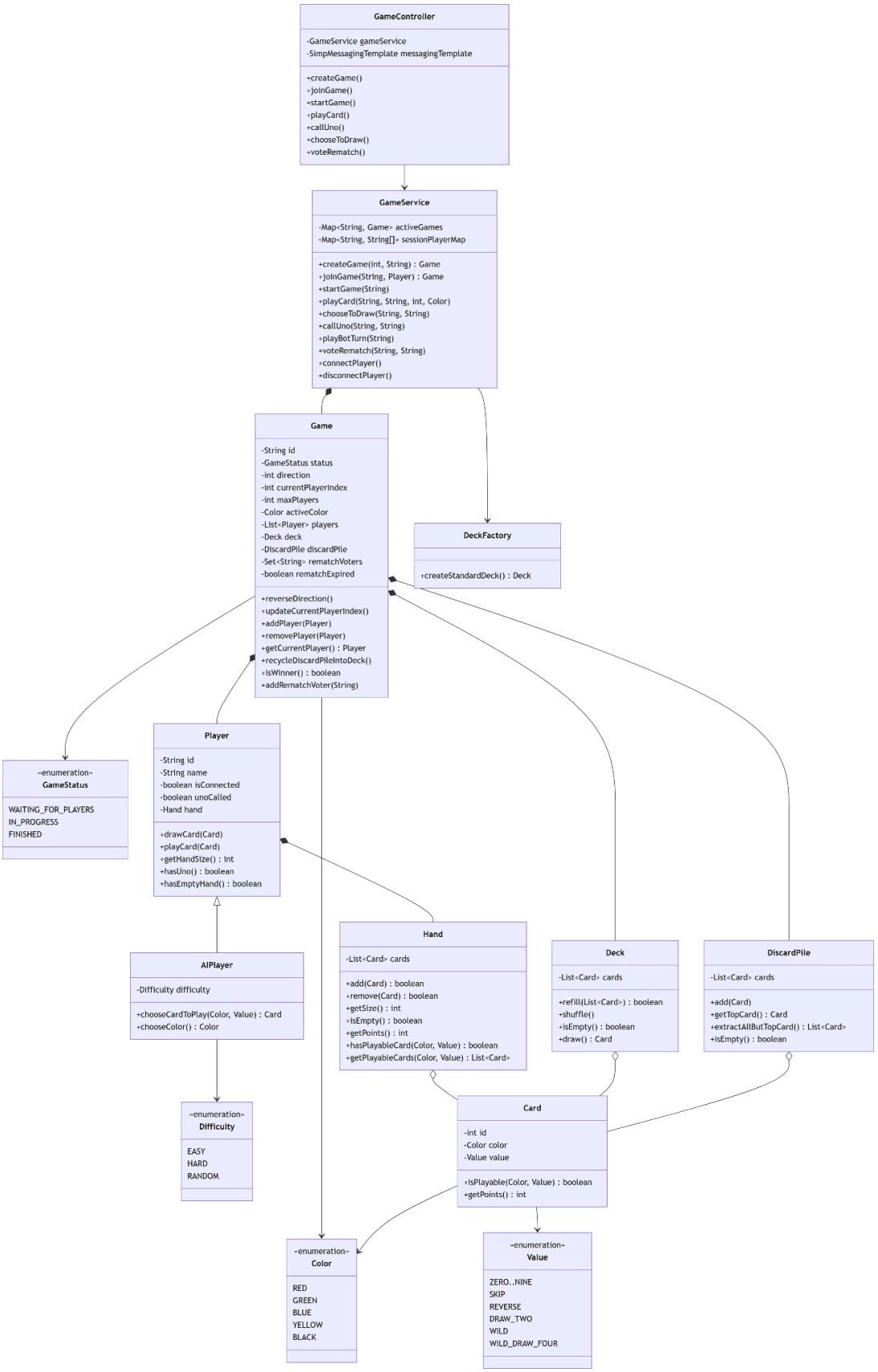
Serveur API (Backend hébergé via Render) : https://uno-upjv.onrender.com

Attention: on utilise des plans gratuits, le serveur s'arrête après 15 min d'inactivité. Il est donc possible que la première requête prenne une à deux minutes, le temps de réveiller le serveur.

2. Dépôt du code source

Code source du serveur et du client : https://github.com/matthieu-comme/uno-upjv

3. Diagramme de classes backend



4. Diagramme de classes Frontend

Diagramme 1 (composants) :

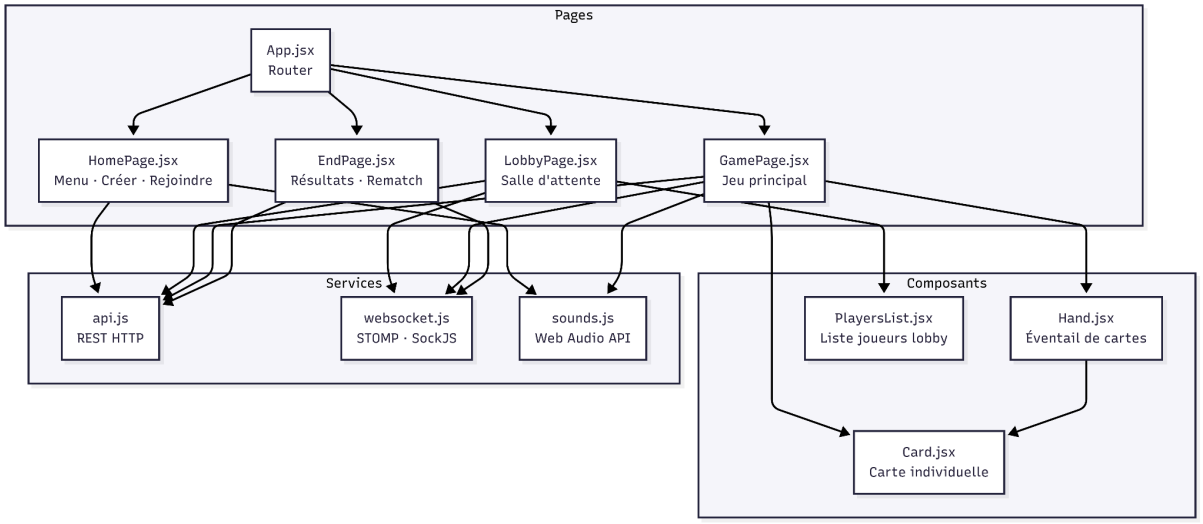
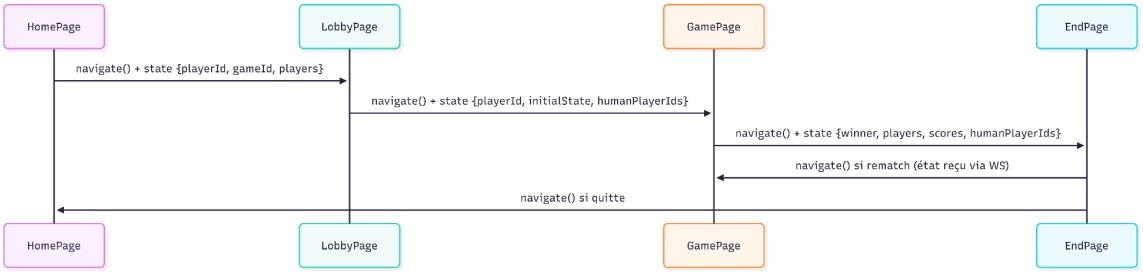



Diagramme 2 (flux de données) :